IC-6600 Principios de Sistemas Operativos · II semestre 2026

midistro: informe final

II Proyecto Programado, Alternativa A: distribución propia basada en Arch Linux

Integrantes: Esteban McKenzie, Ryan Vargas y Geiner Barrantes.
Repositorio: https://github.com/EstebanMckenzie/midistro · **Fecha:** 13 de noviembre de 2026

1. Resumen

midistro es una instalación de Arch Linux con el gestor de ventanas i3, hecha a mano en UEFI y armada en tres capas. La **Capa 1** es la base común: Arch instalado manualmente por cada integrante y un script `install.sh` idempotente que convierte una instalación base limpia en la distribución del equipo. La **Capa 2** implementa siete de los nueve temas de Sistemas Operativos, cada uno con código propio, mediciones y una interpretación del concepto. La **Capa 3** agrega comandos propios que leen `/proc` y `/sys`, y atajos de teclado en i3.

Tema	Responsable	Apoyo	Resultado principal
1 · Procesos y planificación	Esteban	Geiner	Con `nice`, el reparto de CPU coincide con la tabla de pesos del kernel (diferencia menor de 0.1 puntos); con `SCHED_FIFO` el proceso normal queda con 4.7 %
2 · Memoria, swap y zram	Ryan	Esteban	Mismo rendimiento con y sin zram (45.56 contra 45.63 ops/s); con zram el swapfile queda unos 733 MiB menos ocupado, con compresión de 3.8:1
4 · Servicios systemd	Ryan	Geiner	Servicio con timer y cgroup: `CPUQuota=20%` medido entre 18.2 % y 19.9 %; un fallo provocado reinicia el servicio
5 · Concurrencia e IPC	Geiner	Ryan	Sin mutex se pierde el 61 % de los trozos; con mutex, 5 de 5 corridas exactas. Aceleración media x1.96 con 2 CPU
6 · Gestión de paquetes	Geiner	Esteban	Hook de ALPM propio, paquete propio con `PKGBUILD` y verificación de firmas (firma alterada: `BAD signature`)
7 · Proceso de arranque	Esteban	Ryan	Rescate: 4 servicios contra 10. La mejora medida del arranque (−23 %) no se puede atribuir al cambio hecho
9 · Protección y seguridad	Esteban	Geiner	Con firewall de denegación por defecto, el puerto 8080 pasa de *open* a *filtered* en `nmap`

Los temas 3 (sistema de archivos) y 8 (E/S y módulos del kernel) no se eligieron: el primero exige una raíz distinta de ext4, es decir, reinstalar la máquina de cada integrante, y el segundo depende del paso de dispositivos USB en la VM. Todos los datos de este informe provienen de las fichas del repositorio (`docs/`) y de su evidencia (`docs/evidencia/`, `logs/`).

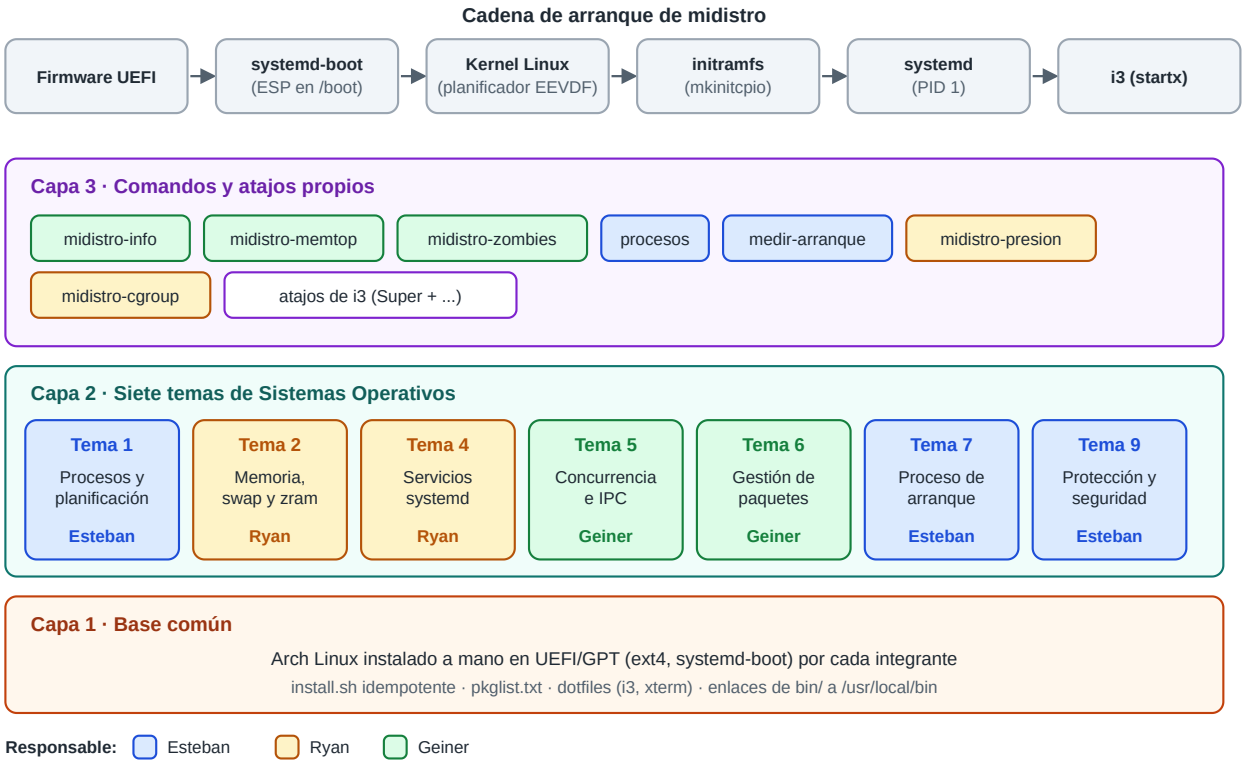


Figura 1. Arquitectura de midistro por capas, con el responsable de cada tema.

2. Capa 1: instalación manual y bootstrap

2.1 Entorno y decisiones de instalación

Dato	Valor
Hipervisor y recursos	VirtualBox sobre Windows; 4 GB de RAM (3888 MB visibles), 2 procesadores
Firmware y disco	UEFI 2.70 (EDK II), Secure Boot desactivado; disco virtual con tabla GPT
Kernel, init y bootloader	Linux 7.2.11-arch1-1, systemd y systemd-boot 262.1
Entorno gráfico	i3 sobre Xorg

La instalación se hizo **sin `archinstall`**, siguiendo la guía de instalación de la Arch Wiki. Cada paso quedó documentado con su porqué en `docs/instalacion.md`. Las decisiones principales fueron estas:

- **Particiones.** GPT con dos particiones: `/dev/sda1` de 1 GiB (EFI, FAT32) montada en `/boot`, y `/dev/sda2` (ext4) como raíz. La ESP se montó en `/boot` y no en `/boot/efi` para que el kernel y el initramfs queden en la partición que `systemd-boot` puede leer. No hay partición de swap: el swap y zram se configuran después, en el tema de memoria, y un archivo de swap es más fácil de cambiar.
- **systemd-boot en lugar de GRUB.** Está diseñado solo para UEFI, viene con el paquete `systemd` y su configuración son dos archivos de texto cortos (`loader.conf` y `entries/arch.conf`), fáciles de explicar y de versionar. GRUB soporta más escenarios, pero aquí no se necesitaban.
- **systemd como init.** Es el init de Arch, y los temas del proyecto dependen de sus herramientas (unit files, cgroups, `systemd-analyze`, timers). Cambiar de init habría obligado a reemplazarlas sin aportar al objetivo del curso.
- **Permisos de `/boot`.** El instalador avisó de un «security hole»: la partición FAT se montaba con `fmask=0022`, legible por cualquier usuario, incluida la semilla aleatoria del kernel. Se cambió a `fmask=0077,dmask=0077`. La consecuencia esperada es que `bootctl status` sin `sudo` da `Permission denied`.

2.2 Elección del entorno gráfico, con datos

Se compararon i3 y XFCE en la misma VM, con una sola terminal abierta y 30 s de espera, usando las columnas `used` y `available` de `free` (y no `buff/cache`, que sube en ambos casos).

Entorno	Memoria usada (MB)	Disponible (MB)	Procesos	Costo sobre la base
Base (sin gráfico)	380	3507	100	n/a
i3	**500**	3387	**111**	+120 MB, +11 procesos
XFCE	615	3273	134	+235 MB, +34 procesos

i3 usa unos 115 MB menos que XFCE y tiene 23 procesos menos, porque es solo un gestor de ventanas: XFCE lanza además panel, gestor de sesión y servicios de energía y notificaciones. Además, toda la configuración de i3 está en un solo archivo de texto (`dotfiles/i3/config`), lo que facilita versionarla y diseñar los atajos. *Limitación:* es una VM, con pocas repeticiones. Una primera lectura de XFCE se descartó por tomarse con más ventanas abiertas que la de i3.

2.3 Bootstrap reproducible: `install.sh`

El script se ejecuta como usuario normal (usa `sudo` cuando lo necesita) y tiene 8 pasos:

Paso	Qué hace
1	Instala el hook de pacman propio *antes* de instalar paquetes, para que la primera transacción ya quede registrada (Tema 6)
2	Instala los paquetes de `pkglist.txt` con `pacman -S --needed --noconfirm`
3	Enlaza los dotfiles (`.xinitrc`, `.Xresources`, configuración de i3); si ya hay un archivo real, lo respalda una sola vez en `.bak`
4	Memoria (Tema 2): instala `zram-generator.conf`, crea el swapfile de 2 GiB y activa zram
5	Instala los unit files propios de `systemd/` en `/etc/systemd/system/`
6	Enlaza los comandos de `bin/` en `/usr/local/bin` y copia `midistro-estado` a `/usr/local/lib/midistro/`
7	Habilita `NetworkManager`, `systemd-timesyncd` y `vboxservice`, y fija el teclado latinoamericano
8	Habilita el timer de estado (Tema 4)

Es **idempotente** gracias a `pacman -S --needed`, `ln -sfn`, `install` y `systemctl enable --now`, que dejan el mismo resultado si se repiten. El firewall del Tema 9 no lo aplica a propósito, porque cierra el acceso por SSH con contraseña; `security/aplicar.sh` se ejecuta aparte.

Prueba final sobre instalación limpia. Se restauró el snapshot previo al escritorio de la VM y se corrió el script dos veces. La primera corrida instaló 154 paquetes (`logs/install-esteban-final.log`); durante la descarga, algunos espejos respondieron `404` para ciertos paquetes y pacman reintentó con otro espejo hasta terminar. La segunda corrida no descargó nada, mostró 25 paquetes «up to date» y no tuvo errores (`logs/install-esteban-final-2da.log`). Después se comprobó que quedaban activos `/swapfile` (2 GiB, prioridad 10), `/dev/zram0` (1.9 GiB, prioridad 100) y `midistro-estado.timer`, y que el servicio terminaba con `status=0/SUCCESS` (`logs/verificacion-final.txt`). Geiner conserva sus propios logs de la primera y segunda corrida (`logs/install-geiner*.log`).



Figura 2. Escritorio de midistro: i3 con `midistro-info` y el monitor de procesos (`procesos`).

3. Capa 2: temas de Sistemas Operativos

Cada tema sigue el mismo orden: qué se implementó, el concepto de Sistemas Operativos, cómo se midió, los resultados y su interpretación, con las limitaciones. La ficha completa de cada uno está en `docs/`.

3.1 Tema 1 · Procesos y planificación (Esteban)

Implementación. `bin/procesos`, una herramienta en Python que lee `/proc` directamente (sin `ps`): lista PID, PPID, estado, nice, política, prioridad de tiempo real y %CPU; cambia el nice con `setpriority` y la política con `sched_setscheduler`. `bin/experimento-prioridades` lanza procesos de CPU puro fijados a un núcleo (`sched_setaffinity`) y compara el CPU recibido con la proporción teórica. El nombre del proceso (`comm`) puede traer espacios o paréntesis, por eso `/proc/PID/stat` se separa con el *último* `)`.

Concepto. Con `SCHED_OTHER`, el nice (−20 a 19) no es una prioridad absoluta: se traduce a un *peso* con la tabla fija `sched_prio_to_weight` (nice 0 → 1024, nice 5 → 335, nice 10 → 110) y cada proceso recibe su peso dividido entre la suma de los pesos. El kernel (7.2.11) usa EEVDF, que reemplazó a CFS en la versión 6.6: ejecuta la tarea elegible con el *deadline* virtual más cercano. `SCHED_FIFO` y `SCHED_RR` son una clase aparte que siempre tiene prioridad sobre la normal; FIFO no tiene rebanada y RR rota cada 100 ms entre iguales. Un *fair server* reserva 50 ms de cada 1000 ms para las tareas normales.

Método. Procesos fijados al núcleo 0 de la VM (2 núcleos), con un bucle de CPU puro. Se mide `utime + stime` de `/proc/PID/stat` durante 10 s; 3 corridas por caso, se reporta el promedio.

Caso	Medido	Pesos	Teórico
nice 0 / nice 0	50.0 % / 50.0 %	1024 / 1024	50.0 % / 50.0 %
nice 0 / nice 5	75.4 % / 24.6 %	1024 / 335	75.3 % / 24.7 %
nice 0 / nice 10	90.3 % / 9.7 %	1024 / 110	90.3 % / 9.7 %
`SCHED_FIFO` 50 contra nice 0	95.3 % (FIFO) / 4.7 % (normal); el normal tuvo 5.1 %, 4.3 % y 4.7 % en las tres corridas		
`SCHED_RR` 50 contra `SCHED_RR` 50	49.1 % / 50.9 %		

Interpretación. Con nice, lo medido y lo teórico difieren en menos de 0.1 puntos porcentuales: nice reparte en proporción, no prioriza de forma estricta. Con tiempo real, la prioridad sí es absoluta. El 5 % que conserva el proceso normal no es casual: aquí `sched_rt_runtime_us` es igual a `sched_rt_period_us` (el límite clásico no está activo), y lo que protege a la tarea normal es el fair server (50 ms por segundo, 5 %). Esta atribución se corrigió durante el trabajo: una primera explicación la atribuía al límite de tiempo real, y al revisar los parámetros del planificador se vio que ese límite no está activo. **Limitaciones:** VM de 2 núcleos con Xorg y otros servicios en el mismo núcleo (explica la variación del caso FIFO); todos los procesos se lanzan desde una sola sesión, porque con `CONFIG_SCHED_AUTOGROUP=y` sesiones distintas diluirían el efecto de nice; el experimento mide reparto de CPU, no latencia.

3.2 Tema 2 · Memoria, swap y zram (Ryan)

Implementación. `systemd/zram-generator.conf` (zram de la mitad de la RAM, `zstd`, prioridad 100); `scripts/configurar-swap.sh`, que crea un swapfile de 2 GiB de forma idempotente (prioridad 10; en Btrfs usa `btrfs filesystem mkswapfile`); `bin/midistro-memoria`, un monitor que lee `/proc/meminfo`, `/proc/swaps`, `/proc/[pid]/stat` y `/sys/block/zram*/mm_stat`; y `scripts/tema2-evidencia.sh`, que reproduce la prueba A/B.

Concepto. Cada proceso ve un espacio de direcciones propio dividido en páginas. Cuando la RAM escasea, el kernel mueve páginas poco usadas al área de intercambio (*swap*) y las trae de vuelta con un fallo de página mayor cuando se necesitan, lo que implica E/S y es lento. *zram* crea un dispositivo de bloque en RAM que comprime las páginas: cambia ciclos de CPU por espacio y evita la E/S de disco. El kernel usa primero el dispositivo de mayor prioridad (zram con 100, el swapfile con 10).

Método. En la misma VM, con 1915.5 MiB visibles (a propósito, para forzar presión de memoria), se ejecutó `stress-ng --vm 1 --vm-bytes 1720M --vm-keep --timeout 300s`, una vez con zram detenida y una con zram activa. El swapfile permanece activo en ambas.

Métrica	Sin zram	Con zram
Resultado de `stress-ng`	13 760 ops; 45.63 ops/s	13 760 ops; 45.56 ops/s
RAM disponible mínima	2.9 MiB	0.0 MiB
Swap ocupada al final	858.6 MiB, toda en el swapfile	833.6 MiB: 708.1 en zram y 125.4 en el swapfile
`pswpin` / `pswpout`	+3871 / +148 370 páginas	+3957 / +169 402 páginas
Compresión de zram (`zramctl`)	n/a	605.3 MiB de datos en 159.1 MiB comprimidos (≈3.8:1)

Interpretación. El rendimiento fue esencialmente igual (diferencia de 0.15 %). Con zram, el swapfile terminó con unos 733 MiB menos de ocupación, mientras que el total de swap fue solo unos 25 MiB menor: zram desplazó parte del intercambio del disco a la RAM comprimida, pero **no demuestra una mejora de velocidad** ni evitó la presión extrema de memoria. Los contadores de fallos son globales y las diferencias de `pgmajfault` no permiten atribuir causalidad a zram. Las cifras de `/proc/swaps` (slots) y de `zramctl` (bytes comprimidos) son medidas distintas y no se restan entre sí. **Limitaciones:** una sola corrida A/B comparada; el código de salida 0 de `stress-ng` no mide latencia interactiva, y el OOM killer debe comprobarse en el journal, no inferirse del uso de RAM.

3.3 Tema 4 · Gestión de servicios systemd (Ryan)

Implementación. `systemd/midistro-estado.service` y `.timer`, y `bin/midistro-estado`, que registra en `/var/lib/midistro/estado.log` la hora, el uptime, los procesos, la memoria disponible, la swap y el espacio en `/`. El servicio es `Type=oneshot`, con `Restart=on-failure`, dependencias `After=`/`Wants=` de journald, límites de cgroup (`CPUQuota=20%`, `MemoryMax=128M`, `TasksMax=32`) y aislamiento (`ProtectSystem=strict`, `ProtectHome=yes`, `PrivateTmp=yes`, `NoNewPrivileges=yes`). El timer lo ejecuta 2 minutos después del arranque y luego cada 10 minutos.

Concepto. Una unidad es un proceso administrado por el sistema operativo: systemd controla su orden de inicio con dependencias, su recuperación con una política de reinicio y su consumo con un cgroup. En cgroup v2, `cpu.max` impone la cuota, `cpu.stat` expone el tiempo de CPU acumulado y `memory.max` el límite de memoria. El timer genera activaciones sin mantener un proceso propio esperando.

Método. `scripts/tema4-evidencia.sh` captura estado, journal y cgroup; durante una carga de CPU toma muestras cada 5 s y calcula el porcentaje como 100 × Δ`usage_usec` / Δtiempo real; después provoca un fallo (salida con código 42) y comprueba que crezca `NRestarts`.

Comprobación	Resultado
Ejecución normal	`status=0/SUCCESS`; pico de memoria 5.3 MiB frente a `MemoryMax=128M`
Cuota de CPU (`cpu.max`)	`20000 100000` (20 % de un núcleo)
CPU en seis ventanas de 5 s	19.1, 19.5, 19.5, 19.2, 19.9 y 18.2 %; máximo 19.9 %; 50 periodos *throttled* por ventana
Fallo provocado	Salida 42, `Result=exit-code`, `NRestarts=1`; reinicio programado en el journal

Interpretación. La carga quedó alrededor del 20 % y el kernel registró *throttling* en cada ventana, coherente con la cuota; la prueba negativa demuestra `Restart=on-failure`. La ejecución periódica evita dejar un daemon residente para una captura breve. **Problema real encontrado:** la primera ejecución falló con `203/EXEC` porque `/usr/local/bin/midistro-estado` era un enlace a un archivo en `/home`, que `ProtectHome=yes` oculta al servicio. Se corrigió ejecutando una copia instalada en `/usr/local/lib/midistro/`, y `install.sh` hace esa copia. El umbral de tolerancia (30 %) es un criterio de la prueba; los datos quedaron por debajo del propio límite de 20 %.

3.4 Tema 5 · Concurrencia e IPC (Geiner)

Implementación. `src/primes_ipc.c` cuenta los primos en [2, N] con un esquema productor/consumidores: un supervisor (padre) crea todo y espera con `waitpid`; un productor corta el rango en trozos de 16 bytes y los escribe en un **pipe**; K trabajadores leen un trozo y suman al resultado global, que vive en **memoria compartida** (`mmap` con `MAP_SHARED|MAP_ANONYMOUS`) protegida por un **mutex** entre procesos (`PTHREAD_PROCESS_SHARED`, robusto); y **señales** (`SIGCHLD`, `SIGTERM`) controlan la vida de los procesos. Dos opciones de demostración: `--no-lock` (provoca la condición de carrera) y `--no-reap` (provoca zombies).

Concepto. Cada proceso tiene un espacio de direcciones aislado, así que cooperar exige un canal del kernel (pipe) o una región compartida. `total += x` es leer-modificar-escribir: si dos procesos se intercalan se pierde una actualización, y se evita con exclusión mutua. Un proceso terminado es *zombie* hasta que su padre llama a `wait()`, y `SIGCHLD` se lo avisa.

Resultados (valores correctos: 200 000 trozos y 148 933 primos; 5 corridas por modo):

Modo	Promedio	Pérdida	Observación
Sin mutex, trozos	77 983	61.0 %	No determinista
Sin mutex, primos	69 449	53.4 %	De 62 513 a 79 407 según la corrida
Con mutex	200 000 y 148 933	0 %	5 de 5 corridas exactas
Aceleración con 2 CPU	x1.96	n/a	Rango de 1.77 a 2.25 en 4 corridas (paralelo 0.666 s, serial 1.321 s en promedio)

Con `strace -f` se observaron `pipe2`, `mmap` compartido, `clone` (el `fork` de glibc), `rt_sigsuspend`, `wait4` y los `write`/`read` de exactamente 16 bytes. Sobre zombies: tras `SIGTERM` o `kill -9` a un trabajador quedan 0; con `--no-reap` quedan 4 mientras el padre vive, y 0 cuando muere y systemd adopta a los huérfanos.

Interpretación y límites. Sin el mutex la carrera es real; con él, el resultado es exacto. En `--no-lock` se llama a `sched_yield()` para ensanchar la ventana de la carrera: la causa es la misma que en un caso real, pero aquí se ve en todas las corridas. Se observaron más hijos recogidos que señales `SIGCHLD` entregadas, porque las señales estándar no se encolan; por eso el handler usa un `while` con `waitpid(-1, ..., WNOHANG)`. La aceleración de x2.25 supera el tope teórico de x2, lo que no es posible con paralelismo real: se explica por el ruido de la VM, porque el tiempo serial y el paralelo se miden en momentos distintos, y por eso se reportan promedio y rango. Si un trabajador muere con `SIGKILL`, el trozo que ya había leído se pierde (1999 de 2000 procesados).

3.5 Tema 6 · Gestión de paquetes (Geiner)

Implementación. (1) Documentación del funcionamiento interno de pacman: bases de datos local y sync, fases de una transacción, verificación de firmas y hooks. (2) Un **hook de ALPM propio** (`pacman/midistro-pkglog.hook`, `PostTransaction`) que registra en `/var/log/midistro-pkg.log` qué paquetes se agregan o quitan y mantiene `/var/lib/midistro/pkglist.txt`. (3) Un **PKGBUILD** que empaqueta el comando `midistro-zombies`.

Concepto. Un gestor de paquetes es una capa sobre el sistema de archivos y la protección: instala archivos de forma controlada, mantiene qué archivo pertenece a qué paquete, y verifica integridad y autenticidad antes de escribir en `/`. También usa exclusión mutua entre procesos: `db.lck` impide dos transacciones simultáneas, el mismo problema del Tema 5 a nivel de archivo.

Medición	Resultado
Base local	414 paquetes, 14 MB, archivos de texto en directorios (`desc`, `files`, `mtree`), no una base SQL
Bases sync	`core.db`: 299 paquetes; `extra.db`: 15 195 paquetes
Árbol de `i3-wm`	10 dependencias directas y 57 transitivas únicas; `ttf-font` es un nombre virtual provisto por `gnu-free-fonts`
Firma válida contra alterada	`Good signature` contra `BAD signature` con un solo byte cambiado (código de salida 1)
Hook en transacciones reales	Registrado en 3 de 3 (`-S tree`, `-R tree`, `-U midistro-zombies`)
Paquete propio	`makepkg` valida el sha256 y crea el paquete; `-Qk`: 3 archivos, 0 faltantes

Interpretación. Una dependencia directa esconde muchas más, y pacman resuelve el grafo completo con los `%DEPENDS%` y `%PROVIDES%` de las bases sync. Con un byte alterado la firma deja de coincidir y la instalación se rechazaría antes de extraer. El hook se ejecuta una vez por transacción, al final, y su prefijo `90-` lo ordena después de los del sistema. El paquete propio aparece como `Validated By: None` (se instaló desde un archivo local sin firma, a diferencia de `bash`); para distribuirlo habría que firmarlo con una llave propia. Empaquetar en lugar de copiar permite consultarlo (`-Qi`, `-Ql`, `-Qo`), verificarlo y desinstalarlo limpiamente. Cada medición de anatomía fue una sola inspección del estado del sistema.

3.6 Tema 7 · Proceso de arranque (Esteban)

Implementación. (1) El mapa de la cadena de arranque: firmware UEFI, `systemd-boot` en la ESP (`/boot`), kernel, initramfs generado por `mkinitcpio` con el hook `systemd`, y `systemd` como PID 1 hasta `graphical.target`. (2) `bin/medir-arranque antes|despues N`, que automatiza `systemd-analyze` (`time`, `blame`, `critical-chain`) y espera a `is-system-running --wait` para no medir a medias. (3) Una optimización: deshabilitar `sshd`. (4) Una entrada

de rescate en `systemd-boot` (`boot/arch-rescue.conf`), idéntica a la normal pero con `systemd.unit=rescue.target`. Como `loader.conf` tiene `editor no` (no se editan parámetros del kernel desde el menú), la entrada tiene que existir como archivo.

Concepto. Cada etapa del arranque carga a la siguiente porque la anterior solo sabe hacer lo mínimo: el firmware ubica un cargador en la ESP, el cargador pone kernel e initramfs en memoria, el kernel monta una raíz temporal donde hay controladores para el disco real, y pasa el control a `init`. systemd no arranca «todo» sino *targets*, metas con dependencias entre unidades: `rescue.target` es una meta mínima y `graphical.target` incluye red, servicios y escritorio.

Método. 3 arranques por fase (antes y después), reiniciando cada vez, misma VM y sin otras ventanas. Para el rescate se compararon los mismos comandos en ambos targets, un minuto después del login.

Fase (promedio de 3)	Kernel	Initrd	Userspace	Total
Antes (con `sshd`)	1.419 s	2.357 s	3.017 s	6.794 s
Después (sin `sshd`)	1.055 s	1.501 s	2.663 s	5.219 s
Diferencia	−0.364 s	−0.856 s	−0.354 s	−1.575 s (−23 %)

Medida	`rescue.target`	`graphical.target`
Targets activos	12	21
Servicios corriendo	4	10
Procesos (`ps -e`, incluye hilos del kernel)	90	145
Red	`enp0s3` abajo (solo `lo`)	`enp0s3` arriba (10.0.2.15)

Interpretación. La mejora de −23 % no se puede atribuir a `sshd`. La mayor parte de la diferencia está en el kernel (−0.364 s) y el initrd (−0.856 s), que se ejecutan antes de que systemd inicie servicios, así que `sshd` todavía no existe en esas fases: la diferencia viene de las condiciones de la medición (caché del disco virtual y arranques consecutivos). En userspace (−0.354 s) tampoco se puede afirmar, porque la dispersión dentro de una misma fase es parecida (de 2.763 s a 3.337 s en «antes»). Con n = 3, el cambio de `sshd` queda dentro del ruido. El rescate, en cambio, sí muestra una diferencia clara: 4 servicios contra 10, 12 targets contra 21 y red apagada. **Un intento fallido que sirvió:** la primera entrada de rescate quedó con `root=UUID=` vacío y el sistema cayó a *emergency mode* («Timed out waiting for device»); se corrigió copiando la entrada normal y cambiando solo el título y la línea `options`. `rescue.target` exige la contraseña de root (`sulogin`) y en midistro root está bloqueado; para la demostración se le puso una contraseña temporal y luego se bloqueó de nuevo con `passwd -l root`. **Limitación:** VirtualBox con EFI no informa los tiempos de firmware ni del cargador.

3.7 Tema 9 · Protección y seguridad (Esteban)

Implementación. `security/aplicar.sh` aplica todo de forma idempotente y valida cada pieza antes de instalarla (`visudo -cf`, `sshd -t`, `nft -c`), revirtiendo si falla. Incluye: el grupo `equipo` y los usuarios `analista` y `invitado` (sin contraseña ni acceso por SSH); un `sudoers` que permite a `analista` solo dos comandos exactos, sin comodines; el directorio compartido `/srv/equipo` con modo `3770` (setgid y sticky) y una ACL que da solo lectura a `invitado`; un firewall `nftables` con `policy drop`; y un endurecimiento de SSH en `sshd_config.d`. AppArmor no se implementó.

Concepto. *Privilegio mínimo*: cada sujeto tiene solo los derechos que necesita, de modo que un error o compromiso queda contenido. Las ACL POSIX añaden permisos por usuario sin meterlo en un grupo, el setgid hace que los archivos nuevos hereden el grupo y el sticky bit impide borrar archivos ajenos. `sudo` es un cambio controlado de dominio de protección: compara el comando completo con sus argumentos, así que un argumento de más deja de coincidir. Un firewall con denegación por defecto actúa como monitor de referencia en la frontera de red y protege incluso a un servicio olvidado.

Método. `nmap -Pn -sV -p 1-10000` desde Windows contra la VM (red *host-only*), antes y después; un servidor `python -m http.server 8080` como puerto «olvidado»; el estado «antes» con `ss`, `sshd -T`, `nft list ruleset` y `sudo -l`; y `security/probar-negativas.sh`, que ejecuta cada caso como el usuario correspondiente y registra salida y código. Antes de endurecer SSH se instaló una llave pública y se tomó un snapshot.

Prueba	Antes	Después
Puerto 22 / puerto 8080 (`nmap`)	open / open	open / **filtered**
Resto del rango	9998 *closed* (reset)	9999 *filtered* (sin respuesta)
`PermitRootLogin` / `PasswordAuthentication`	prohibit-password / yes	**no** / **no**
`MaxAuthTries` / `LoginGraceTime` / `AllowUsers`	6 / 120 / sin definir	3 / 30 / esteban

Prueba negativa o positiva	Resultado
`invitado` escribe o borra en `/srv/equipo`	`Permission denied`; sí puede listar y leer (por ACL)
`analista`: `sudo cat /etc/shadow` y `sudo systemctl restart nftables`	Denegados
`analista`: `journalctl -u sshd.service --no-pager` con un argumento de más / exacto	Denegado / permitido
SSH como `esteban` con llave / sin llave / como `analista` / como `root`	Permitido / `Permission denied (publickey)` en los tres casos denegados

Interpretación. La política `drop` explica el cambio en `nmap`: el servidor del puerto 8080 sigue escuchando, pero el firewall descarta el paquete antes de que llegue a él, y por eso se ve *filtered*. Que los puertos pasen de *closed* a *filtered* demuestra que hay un filtro, aunque también delata que la máquina existe; para ocultarla del todo habría que descartar también el ICMP. El contador final del firewall (20 025 paquetes descartados, unos 2 por cada uno de los 10 000 puertos) confirma que descartó el escaneo. **Limitaciones:** solo TCP, puertos 1 a 10 000, desde un único anfitrión y con `-Pn`; la regla del puerto 22 acepta redes completas (`192.168.56.0/24` y `10.0.2.0/24`) y no una IP exacta, porque el DHCP de VirtualBox puede cambiarla; la llave privada de Windows se generó sin frase de contraseña, aceptable en un laboratorio pero riesgosa; el firewall solo filtra lo que entra; y `sshd` queda deshabilitado en el arranque por una decisión del Tema 7, así que la regla del puerto 22 solo tiene efecto cuando el servicio se inicia a mano. La evidencia de las denegaciones de `sudo` es indirecta: `sudo -l -U analista <comando>` no imprime nada cuando no está permitido, y la denegación se ve por el código de salida 1.

4. Capa 3: comandos y atajos propios

Comando	Autor	Interfaz que usa	Qué hace
`midistro-info` (obligatorio)	Geiner	`/proc` y `/sys`	Kernel, uptime, CPU, GPU y NPU con su driver, memoria, zram, planificador y raíz
`midistro-memtop`	Geiner	`/proc/[pid]/status`, `/proc/meminfo`	Procesos que más memoria usan
`midistro-zombies`	Geiner	`/proc`	Detecta zombies; se empaqueta con `PKGBUILD` (Tema 6)
`procesos`	Esteban	`/proc/[pid]/stat`, llamadas de planificación	Lista, cambia nice y política (Tema 1)
`medir-arranque`	Esteban	`systemd-analyze`	Mide el arranque antes y después (Tema 7)
`midistro-presion`	Ryan	`/proc/pressure/*`, `/proc/vmstat`	PSI de CPU, memoria y E/S, y tasas de fallos de página y swap
`midistro-cgroup`	Ryan	`systemctl show`, cgroup v2	Límites y uso actual del cgroup del servicio de estado

Además, `midistro-memoria` (Tema 2) y `midistro-estado` (Tema 4) sirven de apoyo a las fichas de Ryan. Los atajos obligatorios de i3 son `Super+Enter` (terminal), `Super+W` (cerrar ventana, que desplazó el diseño en pestañas a `Super+T`), `Super+D` (lanzador) y `Super+1` a `Super+9` (espacios de trabajo). Cada integrante diseñó 3 atajos libres que invocan un comando propio o una función de la Capa 2: Esteban, `Super+P`, `Super+Shift+F` y `Super+B` (procesos, firewall y arranque); Geiner, `Super+Ctrl+I`, `M` y `P` (información, memoria y registro de paquetes); y Ryan, `Super+Ctrl+R`, `G` y `E` (presión, cgroup y servicio de estado). La lista completa está en `docs/cheatsheet.md`.

5. Uso de IA

Se usó Claude (Anthropic) como apoyo para guiar comandos, diagnosticar errores, redactar fichas y revisar mediciones. Cada integrante registró sus consultas en `docs/bitacora_ia.md`, que reúne 20 entradas y 18 errores de la IA, numerados

de forma continua. Cada entrada dice qué se pidió, qué se obtuvo y qué se validó, corrigió o descartó. Algunos ejemplos de errores detectados:

- **Atribución incorrecta (Tema 1).** La IA atribuyó el 5 % que conserva un proceso normal frente a `SCHED_FIFO` al límite de tiempo real. Al revisar los parámetros del kernel se vio que ese límite no está activo y que el responsable es el fair server.
- **Comandos que no funcionaban (Tema 9).** La IA propuso filtrar la salida de `sshd -T` con minúsculas, pero imprime las claves con mayúsculas mezcladas; y afirmó una salida de `sudo -l -U` que no es la real. Ambos se corrigieron al ejecutarlos.
- **Verificación de una limpieza (Capa 1).** La IA afirmó que había limpiado un archivo, pero la captura mostró que el temporal se había creado con otro nombre y el original seguía igual.
- **Suposiciones sobre el enunciado y sobre archivos compartidos.** La IA supuso que el comando obligatorio debía ser un paquete, y propuso sobrescribir archivos de otro integrante; se detectó con `git log` antes de aplicarlo.

La lección práctica es que ningún resultado de la IA se aceptó sin ejecutarlo o compararlo con los datos medidos.

6. Limitaciones y lecciones aprendidas

Limitaciones del proyecto

- **Todo se midió en máquinas virtuales** de VirtualBox con 2 núcleos y 1.9 a 3.9 GiB de RAM, sobre un anfitrión que comparte CPU y caché de disco. Los tiempos absolutos varían entre corridas, y las conclusiones se apoyan en proporciones o en diferencias grandes.
- **Pocas repeticiones** (3 a 5 por caso). En el Tema 7 esto impidió atribuir un cambio de −23 % a su causa, y en el Tema 2 la comparación A/B es de una corrida por modo.
- **Alcance acotado.** El escaneo del Tema 9 cubre solo TCP en los puertos 1 a 10 000 desde un anfitrión; AppArmor no se implementó; los temas 3 y 8 no se eligieron.

Lecciones aprendidas

- **Medir antes de afirmar.** Tres resultados contradijeron la intuición inicial: deshabilitar `sshd` parecía acelerar el arranque pero el cambio está dentro del ruido (Tema 7); zram no mejoró el rendimiento aunque sí redujo el uso del swapfile (Tema 2); y una aceleración de x2.25 con 2 CPU es imposible y delata el ruido de la VM (Tema 5). Documentar esos límites fue más valioso que reportar solo las cifras favorables.
- **Reproducibilidad.** El bootstrap tenía que funcionar sobre una instalación limpia, no solo en la máquina de trabajo: ahí aparecieron problemas que el uso diario ocultaba, como el hook de pacman que debía instalarse *antes* de los paquetes, o el servicio que fallaba con `203/EXEC` porque `ProtectHome=yes` oculta `/home`.
- **Un solo dueño para `install.sh`.** Evitó conflictos de fusión; los demás aportaron sus archivos en carpetas propias. Los commits pequeños y frecuentes, cada uno con su usuario, hacen que el historial refleje el reparto.
- **Entender cada parte antes de integrarla.** La defensa pide explicar y modificar el propio componente y responder sobre el de otro; por eso cada tema tiene un apoyo y cada ficha explica el concepto, no solo el resultado.

7. Estructura de la evidencia

El repositorio contiene `install.sh` y `pkglist.txt`; `bin/` con los comandos; `dotfiles/` (i3, xinitrc, Xresources); `security/`; `pacman/`; `src/`; `systemd/`; `boot/`; `scripts/` (que reproducen la evidencia de los Temas 2, 4, 5 y 6); `logs/`; y `docs/` con la bitácora de instalación, la matriz de responsabilidades (`responsabilidades.md`), la bitácora de IA, el cheatsheet, la documentación de comandos, las fichas de los siete temas y la evidencia en `docs/evidencia/`.

Referencias

- A. S. Tanenbaum y H. Bos, *Modern Operating Systems*, 4.ª ed. Pearson, 2015.
- A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.ª ed. Wiley, 2018.
- J. H. Saltzer y M. D. Schroeder, «The Protection of Information in Computer Systems», *Proceedings of the IEEE*, vol. 63, n.º 9, 1975.

- I. Stoica y H. Abdel-Wahab, *Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation*, informe técnico, Old Dominion University, 1995.
- Documentación del kernel Linux: `Documentation/scheduler/sched-eevdf.rst`; *Control Group v2*.
- M. Kerrisk, páginas del manual de Linux: `pipe(7)`, `waitpid(2)`, `signal(7)`, `sched(7)`, `chrt(1)`, `taskset(1)`.
- Páginas del manual: `systemd.service(5)`, `systemd.timer(5)`, `systemd.resource-control(5)`, `systemd.exec(5)`, `bootup(7)`, `systemd.special(7)`, `systemd-analyze(1)`, `sudoers(5)`, `acl(5)`, `nft(8)`, `sshd_config(5)`, `alpm-hooks(5)` y `PKGBUILD(5)`.
- Arch Wiki: «Installation guide», «systemd-boot», «mkinitcpio», «Pacman», «Pacman/Package signing», «Sudo», «Access Control Lists», «nftables», «OpenSSH» y «Zram».